

Edit Distance

- SUNNY, SNOWY
- Insertions, Deletions, Substitutions

- Delete U → SNNY
- N → O → SNOY
- Insert W → SNOWY

S U N N - Y
S - N O W Y

last column $\boxed{s_1[-1]}$ or $s_2[-1]$ or $s_1[-1]$
Deletion Insertion Substitution/
Match

$$ED(s_1[:i-1], s_2) + 1$$

$$ED(s_1, s_2[:i-1]) + 1$$

$$ED(s_1[:i-1], s_2[:i-1]) + \begin{pmatrix} \text{last char match} \\ ? & 0 \\ : & 1 \end{pmatrix}$$

DP solution

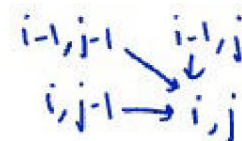
- $dp[i][j]$: Edit distance between $s_1[:i]$ & $s_2[:j]$
prefix of s_1 of len i

- Base case: $dp[i][0] = i$
 $dp[0][j] = j$

Recurrence relation: $dp[i][j] = \min \begin{cases} dp[i-1][j] + 1 \\ dp[i][j-1] + 1 \\ dp[i-1][j-1] + \underbrace{0/1}_{\text{band on } s_1[i-1] == s_2[j-1]} \end{cases}$

- Order of filling:

- row by row filling each row from left to right

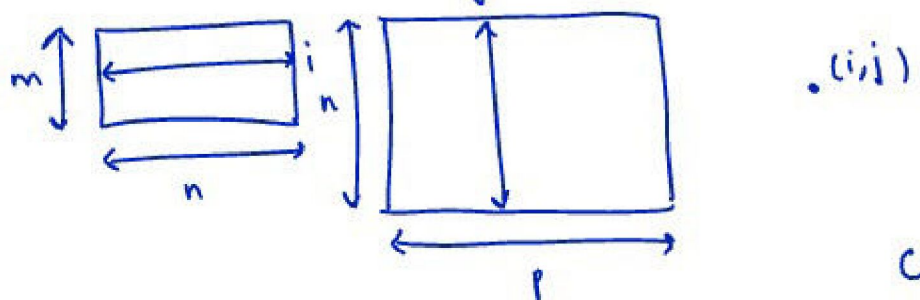


- Final answer: $dp[m][n]$
 $len(s_1) = m$
 $len(s_2) = n$

- RT: #subproblems: $(m+1)(n+1)$
Work per subproblem: $\Theta(1)$
Post processing: $\Theta(1)$
RT: $\Theta(mn)$

Chain Matrix Multiplication:

$$A_{m \times n} \times B_{n \times p} = C_{m \times p}$$



$$\begin{matrix} 0 & 1 \\ [a_0 & a_1 \dots a_n] \end{matrix} \begin{matrix} 0 & 1 \\ [b_0 & b_1 \dots b_n] \end{matrix}$$

Dot product: $a_0 b_0 + a_1 b_1 + \dots + a_n b_n$

RT: for r in C
 for c in C
 $C[r][c] = A[r] \cdot B[c]$
 $\theta(n)$
 Dot product of 2 'n' sized vectors

m times p times

RT: $\theta(mnp)$

Chain Matrix Multiplication

$$(A \times B) \times C = A \times (B \times C)$$

Eg: $A: 50 \times 20$
 $B: 20 \times 1$
 $C: 1 \times 10$
 $D: 10 \times 100$

$A \times B \times C \times D$

$$A((BC)D)$$

$$BC \rightarrow 20 \cdot 1 \cdot 10 = 200 \text{ ops}$$

$$(BC)D \rightarrow 20 \cdot 10 \cdot 100 = 20000 \text{ ops}$$

$$A((BC)D) \rightarrow 50 \cdot 20 \cdot 100 = 100000 \text{ ops}$$

total: 120200 ops

$$(AB)(CD)$$

$$AB \rightarrow 50 \cdot 20 \cdot 1 = 1000 \text{ ops}$$

$$CD \rightarrow 1 \cdot 10 \cdot 100 = 1000 \text{ ops}$$

$$(AB)(CD) \rightarrow 50 \cdot 1 \cdot 100 = 5000 \text{ ops}$$

7000 ops

$$(A(BC))D$$

$$BC \rightarrow 200$$

$$A(BC) \rightarrow 50 \cdot 20 \cdot 10 = 10000$$

$$(A(BC))D \rightarrow 50 \cdot 10 \cdot 100 = 50000$$

60200 ops

Greedy approach:

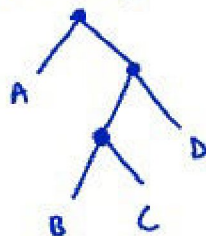
BC 20x10 : 200

$A(BC)$

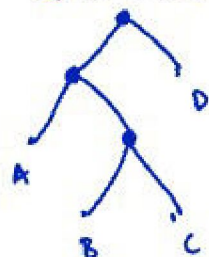
$(A(BC))D$

Not optimal.

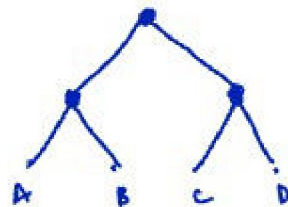
A ((BC)D)



$(A(BC))D$



$(AB)(CD)$



Matrices: $A_1, A_2, A_3, \dots, A_{n-1}, A_n$
 dimensions: $m_0, m_1, m_2, m_3, \dots, m_{n-1}, m_n$

$A_i : m_{i-1} \times m_i$

$A_1 : m_0 \times m_1$

$A_2 : m_1 \times m_2$

$A_3 : m_2 \times m_3$

$dp[i][j]$: min steps for multiplying

$A_i \times A_{i+1} \times \dots \times A_j$

Base case: $dp(i, i) = 0$

Recurrence relation: $dp[i][j] = \min_{i \leq k < j} \{ dp[i][k] + dp[k+1][j] + m_{i-1} m_k m_j \}$

$A_i \times A_{i+1} \times \dots \times A_{j-1} \times A_j$

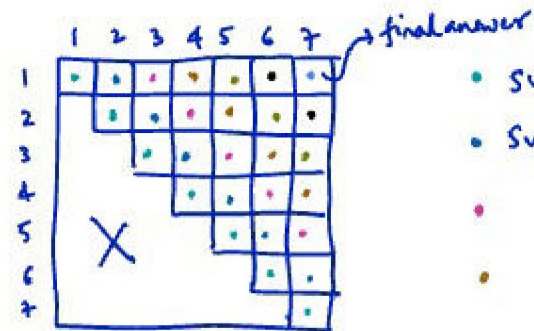
$(A_i \times A_{i+1} \times \dots \times A_k) (A_{k+1} \times A_{k+2} \times \dots \times A_j)$

k goes from i to j-1

Dimensions:

$A_i \times A_{i+1} \times \dots \times A_k \rightarrow m_{i-1} \times m_k$
 $\downarrow \qquad \qquad \qquad \downarrow$
 $m_{i-1} \times m_i \qquad \qquad m_{k-1} \times m_k$

$A_{k+1} \times A_{k+2} \times \dots \times A_j \rightarrow m_k \times m_j$



$A_1 A_2 A_3 A_4 A_5 A_6 A_7$

min #ops: $dp[1][7]$

- subproblem of size 1
- subproblems of size 2
- " " 3
- " " 4
- " " 5
- " " 6
- " " 7

• Order of filling:

fill the array from smallest subproblems (size 1 $\rightarrow$ diagonal) to largest subproblems as any subproblem needs only solutions to smaller subproblems

• Final answer: $dp[1][n]$

• RT: #subproblems: $\frac{n(n+1)}{2} = \Theta(n^2)$

work per subproblem: $O(n)$

post processing: $\Theta(1)$

RT: $O(n^2)$

$$\underbrace{n(1)}_{\text{Base Case}} + (n-1)\underline{1} + (n-2)\underline{2} + (n-3)\underline{3} + \dots + (1)\underline{(n-1)}$$

Base Case

$A_i A_{i+1}$

k goes from $\begin{matrix} i & \text{to} & j-1 \\ i & \text{to} & i \end{matrix}$

Maximum Subarray sum

nums -2 1 3 4 -1 2 1 -5 4

$\ln(\text{nums}) = n$

DP solution:

• $dp[i] :=$ Max subarray sum ending at index i

Base case: $dp[0] = \text{nums}[0]$

Recurrence: $dp[i] = \max\{\text{nums}[i], \text{nums}[i] + dp[i-1]\}$
relation

Order of filling: left to right

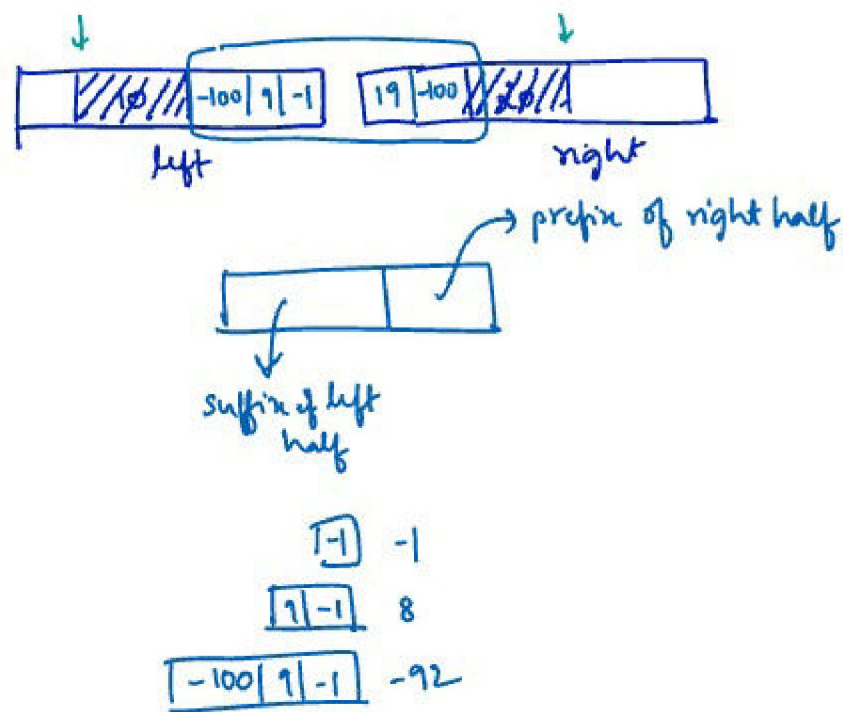
Find answer: $\max\{dp[i]\}$

RT: #subproblems = n

work per subproblem: $\Theta(1)$

post processing: $\Theta(n)$

RT = $\Theta(n)$



$T(n)$: Time for finding max subarray sum on size 'n' array

$$T(n) = 2T(n/2) + \underline{\underline{\Theta(n)}}$$

$$\downarrow$$

$$T(n) = \Theta(n \log n)$$

Recurrence relation for RT

def f(nums): if nums is empty: return 0, 0, 0, 0
 left-total, left-max-sum, left-max-prefix, left-max-suffix = f(left)
 right-total, right-max-sum, right-max-prefix, right-max-suffix = f(right)

$$\text{max-sum} = \max \{ \text{left-max-sum}, \text{right-max-sum}, \text{left-max-suffix} + \text{right-max-prefix} \}$$

$$\text{max-prefix-sum} = \max \{ \text{left-max-prefix}, \text{left-total} + \text{right-max-prefix} \}$$

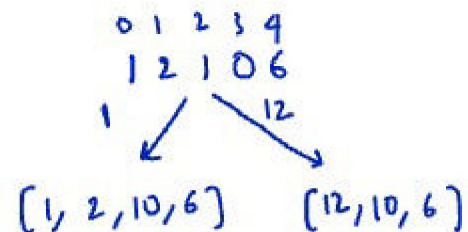
$$\text{max-suffix-sum} = \max \{ \dots \}$$

return left-total + right-total,
 max-sum,
 max-prefix-sum,
 max-suffix-sum

Recurrence for RT:

$$T(n) = 2T(n/2) + \Theta(1)$$

$$\Rightarrow T(n) = \Theta(n)$$



106 → [10, 6]
2106 → [2, 10, 6]

n : #chars in the given string
 m : #words in dictionary

$dp[i]$: #ways $s[i:]$ could be decoded

Base case: $dp[n] = 1$
 $dp[n-1] = \text{num}[n-1] > 0$
 ? 1
 : 0

Recurrence relation:

numbers $x_1 = \text{int}(s[i])$
 $x_2 = \text{int}(s[i:i+2])$
 $dp[i] = ((x_1 > 0) ? dp[i+1] : 0)$
 $+ ((10 \leq x_2 \leq 26) ? dp[i+2] : 0)$

Order of filling: right to left

Final answer: $dp[0]$

RT: #subproblems = $n+1$
 work per subproblem = $\Theta(1)$
 postprocessing = $\Theta(1)$
 RT: $\Theta(n)$

$dp[i]$: T/F based on whether or not
 $s[i:]$ has a valid word break

$dp[n] = \text{True}$

$dp[i] = \text{True}$ if $\exists j$ st. $s[i:j]$ is a valid word
 and $dp[j]$ is true

False otherwise

Order of filling: right to left

Final answer: $dp[0]$

RT: #subproblems = $n+1$
 work per subproblem = $\Theta(1)$ if
 1. we have a set of dictionary words
 2. max word length is 20
 (LC constraint)
 preprocessing = $\Theta(m)$
 RT = $\Theta(n+m)$ to create the set
 ($\Theta(n)$ otherwise)

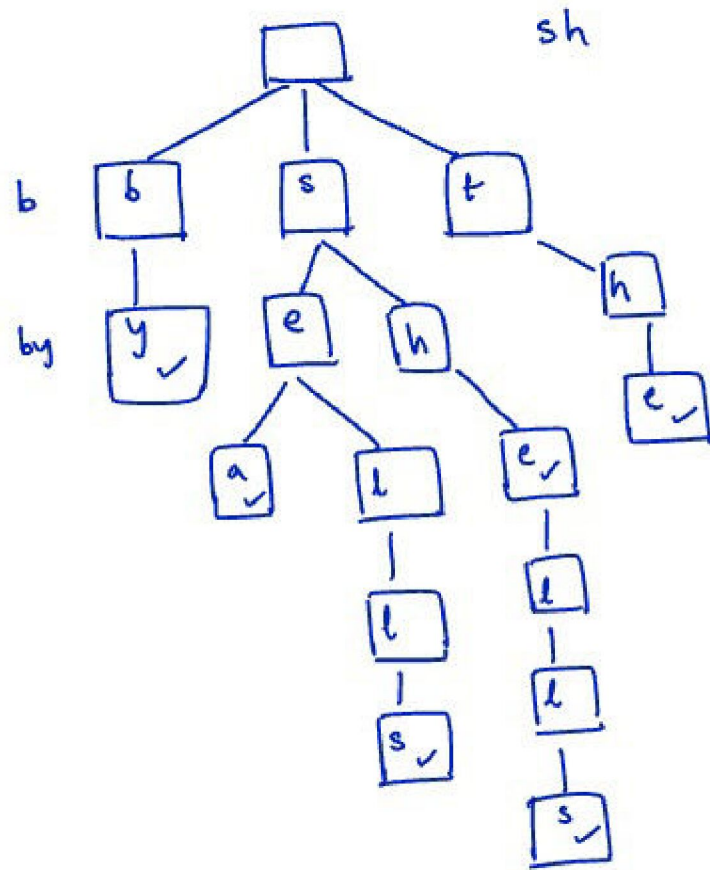
$dp[i]$ potentially depends upon
 $dp[i+1], dp[i+2], \dots, dp[i+20]$

Words dictionary: [ABCDEFGHJJ, DEF]

DEFCGH...

Retrieval → Trie

. Dictionary of words: by, sea, sells, she, shells, the



```
class TrieNode {  
    HashMap<Character, TrieNode> children;  
    boolean isWord;  
}
```


Stock Buy & Sell

with[i]: max profit at end of day i
where we hold the stock at the end of day i

without[i]: max profit at end of day i
where we don't hold the stock at the end of day i

$$\text{with}[0] = -\text{prices}[0]$$

$$\text{without}[0] = 0$$

$$\text{with}[i] = \max \{ \text{with}[i-1], \text{without}[i-1] - \text{prices}[i] \}$$

Two cases:

- Buy
- Continue holding the existing stock

$$\text{without}[i] = \max \{ \text{without}[i-1], \text{with}[i-1] + \text{prices}[i] \}$$

Two cases

- Continue not holding the stock
- Sell

Order of filling: from left to right

$$\text{Final answer: } \max \{ \text{with}[n-1], \text{without}[n-1] \}$$

or

$$\text{without}[n-1]$$

RT: #subproblems = n

post processing = $\Theta(1)$

Work per subproblem = $\Theta(1)$

$\Rightarrow \text{RT} = \Theta(n)$

Transaction fee after selling

$$\text{with}[i] = \max \{ \text{with}[i-1], \text{without}[i-1] - \text{prices}[i] \}$$

$$\text{without}[i] = \max \left\{ \text{without}[i-1], \begin{array}{l} \text{with}[i-1] + \text{prices}[i] \\ - \text{fee} \end{array} \right\}$$

$$\text{with}[0] = -\text{prices}[0]$$

$$\text{with}[1] = \max \{ -\text{prices}[0], -\text{prices}[1] \}$$

$$\text{without}[0] = 0$$

$$\text{without}[1] = \max \{ 0, \text{prices}[1] - \text{prices}[0] - \text{fee} \}$$

Cooldown of 1 day (at least)
after selling before buying again.

$$\text{with}[i] = \max \{ \text{with}[i-1], \text{without}[i-2] - \text{prices}[i] \}$$

$$\text{without}[i] = \max \{ \text{without}[i-1], \text{with}[i-1] + \text{prices}[i] \}$$

$$\text{with}[0] = -\text{prices}[0]$$

$$\text{with}[1] = \max \{ -\text{prices}[0], -\text{prices}[1] \}$$

$$\text{without}[0] = 0$$

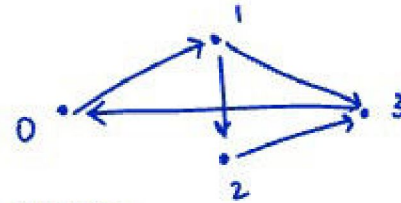
$$\text{without}[1] = \max \{ 0, \text{prices}[1] - \text{prices}[0] \}$$

Graphs $\rightarrow \# = V$ $\rightarrow \# = E$

- Vertices, Edges connecting them
(directed $\Rightarrow$ directed graph
undirected $\Rightarrow$ undirected graph)

- Three representations

- Adjacency matrix
- Adjacency list
- Edge list



(Assume that there are no self loops & no parallel edges)

Adj Matrix

	0	1	2	3
0	0	1	0	0
1	0	0	1	1
2	0	0	0	1
3	1	0	0	0

$$[i][j] = 1 \iff i \rightarrow j$$

Space required: $\Theta(V^2)$

Adj List

0 : 1 $\rightarrow$
1 : 2 $\rightarrow$ 3 $\rightarrow$
2 : 3 $\rightarrow$
3 : 0 $\rightarrow$

Space: $\Theta(V + E)$

Edge List

[(0,1), (1,2), (3,0),
(1,3), (2,3)]

Space: $\Theta(E)$

- Indegree: #edges coming into a vertex
- Outdegree: #edges going out of a vertex

	Adj Matrix	Adj List	Edge List
(u,v) edge present	$\Theta(1)$	$O(\text{outdegree}(u))$	$O(E)$
get neighbours of u	$\Theta(V)$	$\Theta(\text{outdegree}(u))$	$\Theta(E)$
compute outdegree of u	$\Theta(V)$	$\Theta(\text{outdegree}(u))$	$\Theta(E)$
compute indegree of u	$\Theta(V)$	$O(V + E)$	$\Theta(E)$

- Directed graph of V vertices
 - max edges = $V(V-1) = \Theta(V^2)$

- Undirected graph of V vertices
 - max edges = $\frac{V(V-1)}{2} = \Theta(V^2)$,

- Undirected graph
 - $\sum_{u \in V} \text{degree}(u) = 2E$

- Directed graph
 - $\sum_{u \in V} \text{indegree}(u) = E = \sum_{u \in V} \text{outdegree}(u)$

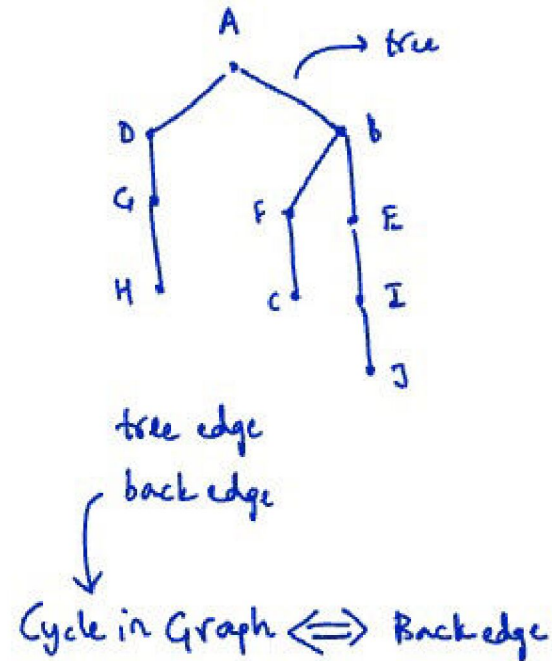
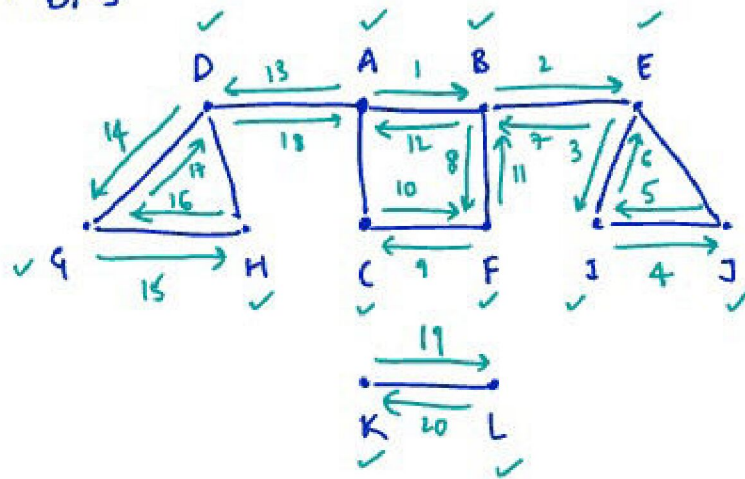
} Handshake lemma

- Sparse graphs, $E = O(V) \rightarrow$ prefer Adj List
Dense graph otherwise $\rightarrow$ consider Adj Matrix
- Edge List is better in certain situations
Eg: Kruskal's Minimum Spanning Tree algo

Graph Traversal

- DFS - Depth First Search
- BFS - Breadth First Search

DFS



explore(u):

visited[u] = true

for each neighbour v of u:

if not visited[v]:

explore(v)

dfs(G):

mark all vertices as not visited; cc = 0

for each vertex u in G.V:

if not visited[u]:

explore(u)

cc++

clock = 0

explore(u):

visited[u] = True

// Previsit code - runs when we first encounter the vertex

clock++; pre[u] = clock

for each neighbour v of u:

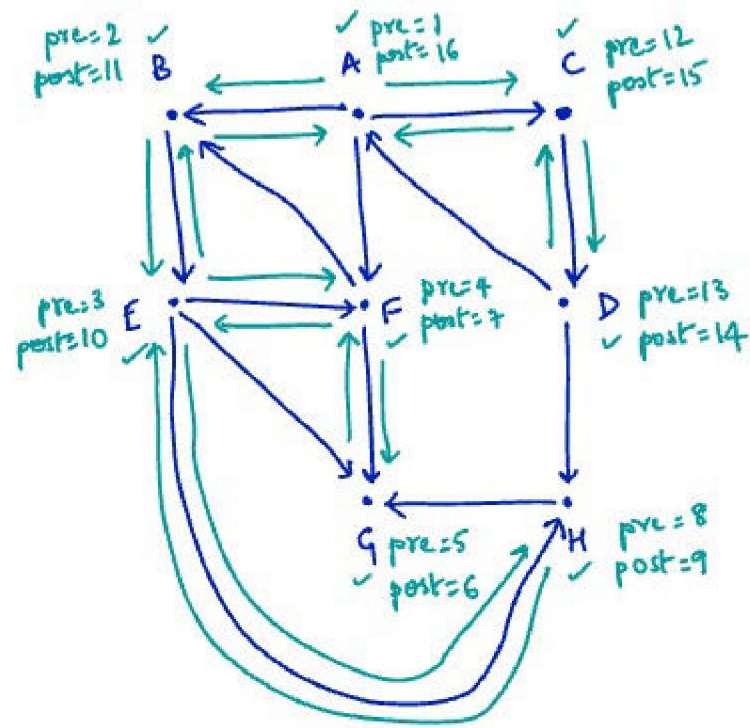
if not visited[v]:

explore(v)

// Post visited code

clock++; post[v] = clock

Directed graphs



For any pair of nodes

$$\begin{bmatrix} [&] \\ u & v \end{bmatrix} \quad \begin{bmatrix} [&] \\ v & u \end{bmatrix} \quad \begin{bmatrix} [&] \\ u & v \end{bmatrix} \quad \begin{bmatrix} [&] \\ v & u \end{bmatrix}$$

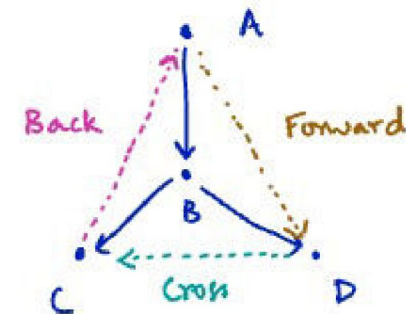
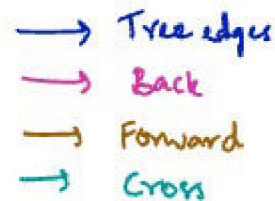
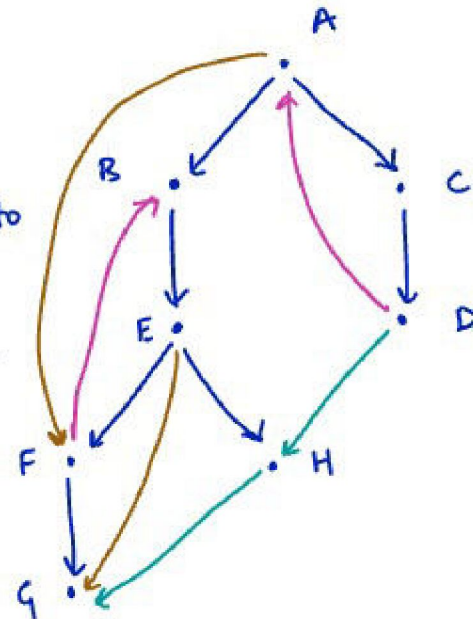
$$\begin{bmatrix} [&] \\ u & v \end{bmatrix} \quad \times$$

DFS Tree

Forward edge: edge from a node to a non-child descendant

Back edge: edge from a descendant to an ancestor

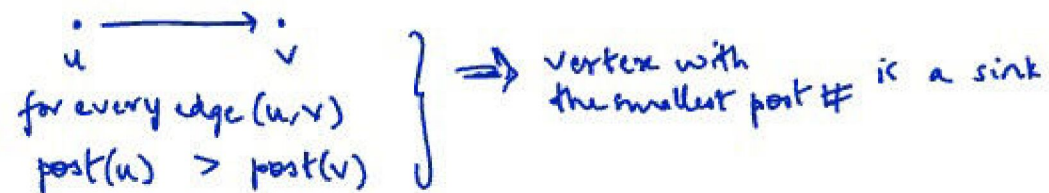
- $u \rightarrow v$ is a back edge
 $\Leftrightarrow \text{post}(u) < \text{post}(v)$



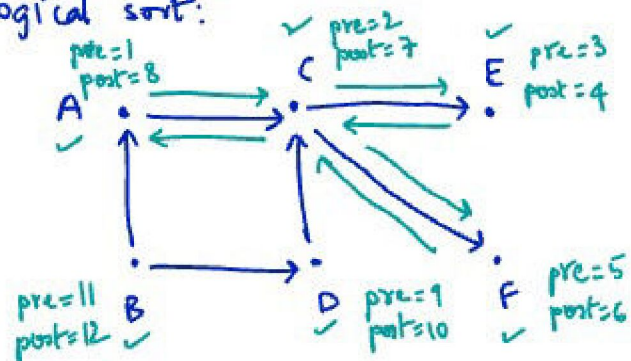
Back edge $\Leftrightarrow$ Cycle in graph

Directed Acyclic graphs

- Directed graphs that don't have a cycle



Topological sort:



12	10	8	7	6	4
B	D	A	C	F	E

vertices in decreasing order of DFS post #

B, A, D, C, E, F

B, D, A, C, F, E

...